

A Novel Approach to Object Outline Detection

Aashutosh Singh Baghel

Department of Computer
Science and Engineering

Samrat Ashok Technological
Institute

Vidisha, Madhya Pradesh,
India

(AICTE approved. Affiliated
to RGPV, Bhopal)

aashutosh28cse004@satiengg.
in

Kuldeep Patel

Department of Computer
Science and Engineering

Samrat Ashok Technological
Institute

Vidisha, Madhya Pradesh,
India

(AICTE approved. Affiliated
to RGPV, Bhopal)

kuldeep28cse067@satiengg.in

Mrinal Rangare

Department of Computer
Science and Engineering

Samrat Ashok Technological
Institute

Vidisha, Madhya Pradesh,
India

(AICTE approved. Affiliated
to RGPV, Bhopal)

mrinal28cse078@satiengg.in

ABSTRACT – *Edge detection tends to be one of the most important aspects of image analysis and interpretation. The information gathered from this helps run machinery and robotics on a large scale. Extracting this information can be made simpler by identifying the boundaries of objects within the image. This paper will aim to study a new, novel solution for processing edges in a given image. General algorithm is discussed. Setbacks in this approach, along with their suggested solutions are also discussed.*

I. INTRODUCTION

An **image** is a visual representation. It can be two-dimensional, such as a drawing, painting, or photograph, or three-dimensional, such as a carving or sculpture. Images may be displayed through other media, including a projection on a surface, activation of electronic signals, or digital displays; they can also be reproduced through mechanical means, such as photography, printmaking, or photocopying.[1]

A digital image comprises of a large grid of pixels, where each pixel has 3 channels of data (4 channels if codec supports transparency). Every channel holds some value for every pixel. These are the rgb/cmy channels.

II. ALGORITHM PIPELINE

i. Conversion of image data.

Every pixel has 3 channels that define its colour. This colour value is represented in integer values.

The image is loaded into memory, and every pixel is interpreted and stored as a numerical digit.

- 0-255 channel range for 8 bit images*¹
- 0-65535 channel range for 16 bit images

Pixel value = $\Sigma(\text{individual channel values}) \dots(1)$

For the record let us call these values **K**. These **K** values are saved into an array which is equal to the size of the image.

ii. Near pixel differences.

For a particular pixel, the difference between its **K** value and the **K** value of nearby pixels is calculated*². The sum of these differences is stored for every pixel in a similar fashion to how the **K** values are stored. This process is to be repeated for every pixel in the image. This results in a pixel grid, where the value for every pixel is equal to the variance from the nearby pixels. This grid is used to create a set variance gradient map of sorts. Let us call the sum of differences of **K** of nearby pixels as **Ω**.

1. Number of pixels used for finding mean difference is tied to image compression level.

2. Result images are 8 bit colour. 8 bit colour images allow for better memory usage at the cost of quality that 16 bit images offer.

iii. Filteration.

For every pixel where Ω is larger than a set threshold, the edge state of the pixel is set to true. If Ω does not exceed the threshold, the edge state is set to false. For pixels on edge where, many of expected nearby pixels do not exist, the value of Ω is corrected by multiplying a constant θ . This constant can be calculated as follows:

$$\theta = \frac{\Sigma \Omega}{xy} \times z \quad \dots(2)$$

- x = horizontal pixels in image
- y = vertical pixels in image
- z = number of missing pixels

iv. Correcting odd exposure and lens glares.

When images are taken in odd*³ conditions and uneven surfaces, images get artefacts as follows:

- Lens glares
- Light smudges

These artefacts can be accounted for and corrected by multiplying the whole image by a simple gradient mask. This mask can be calculated by concatenating the diagonal Ω values. This gives us two gradient masks, one for each diagonal. Ω per pixel can then be standardised as follows:

$$\Omega_{(x,y), \text{corrected}} = (\Omega_{(x,y), \text{original}})(\alpha) \quad \dots(3)$$

- $\Omega_{(x,y), \text{corrected}}$ = corrected promixity difference
- $\Omega_{(x,y), \text{original}}$ = original promixity difference
- α = gradient mask at current column/row

Final image can then be formulated on the basis of Ω exceeding threshold value. If exceeds, the particular pixel can be represented as black and if it does not exceed, it can be represented as white on the final image.[2]

III. RESULTS

The following images are 8 bit png files of size 512x512. No transparency is present.

The program was written in python3. Images provided below are:

- Original image
- Image with near pixel differences
- Image with contour masking (I.iv)

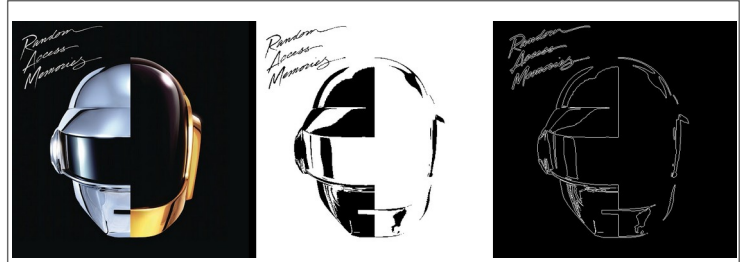


Figure 1: A helmet image passed through the program with differences and correction

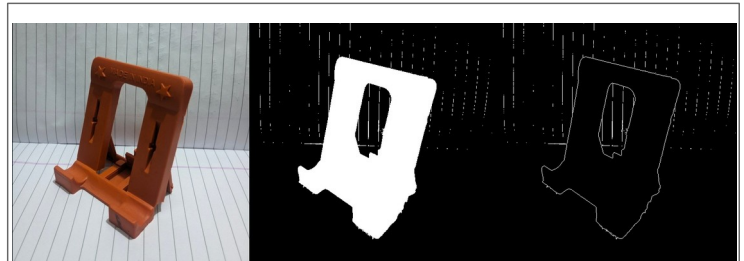


Figure 2: Original, Differences, and Correction

As apparent from the results, the algorithm can still be optimized. Standard algorithms such as Roberts, Sobel, Prewitt, Laplacian and Canny[3] offer better results in most cases, but can be matched in some cases.

IV. PROBLEMS OF THIS APPROACH

i. High computational requirement

The algorithm does not focus on the compressed blocks of image but rather on individual pixels. Due to this inherent flaw, the computation power required is very high. This limits usage in video feeds which require just-in-time conversion. Ideally the image block should be processed without full decoding_[see V.ii][4]. This however would pose the problem of re-writing the program for every codec.

ii. High bitrate requirement

Increasing the number of nearby pixels to calculate $K_{[see V.ii]}$, increases the computational load by factor of block size. On low bitrate images, this

³. Odd conditions refer to unstable camera position, uneven lighting, poor camera resolution

offers diminishing returns as compared to other prevalent methods for edge detection. As a result, the model works faster on high bitrate image while it takes more compute to process low bitrate images.

V. PROPOSED SOLUTIONS

i. Pre-application of filter

Usage of filters that enhance contrast of image and accentuate colours before processing can help make edge detection easier.

ii. Increaseing number of pixels considered for difference.

Lower bitrate images and lossy codecs exhibit significant ‘blockiness’. The ‘image block’ refers to a small, square section of pixels, which are treated as a single unit during the compression process. A completely lossless png image has a block size of 1x1 pixels. The boundaries of these blocks can create unwanted edges in the image.

This can be countered by increasing the number of pixels used for calculating Ω (sum of rgb differences). This lowers the probability of detection of ‘block’ edges. This however, increases the computation load_[see IV.i]. It could be possible to work around this by processing the blocks themselves instead of the pixels themselves_{[see [5]]}.

VI. ACKNOWLEDGEMENTS

I would like to express my sincere gratitude to Dr. Kanak Saxena, Head of the Department of Computer Science and Engineering, for their constant support, guidance, and encouragement throughout the course of this research. Their valuable insights and constructive feedback have been instrumental in shaping this project.

I am also deeply thankful to Dr. Neelesh Mehra and Mr. Mukesh Azad for providing post-haste clarification. I am also deeply thankful to the Technovision Committee for providing me with the opportunity to present my research at a reputable platform.

Without the support of them, this research would not have been possible.

VII. REFERENCES

- [1] *"Meaning and Function of a Picture, published by Taylor & Francis, Ltd. on behalf of the Mathematical Association of America, DOI: 10.2307/2301228on Jstor.Org". JSTOR 2301228*
- [2] Dim, Jules R.; Takamura, Tamio (2013-12-11). *"Alternative Approach for Satellite Cloud Classification: Edge Gradient Application"*
- [3] https://www.projectrhea.org/rhea/index.php/Edge_Detection_with_Gaussian_Blur by Buyue Zhang
- [4] Martin KRÁLIK, Libor LADÁNY *"Canny Edge Detector Algorithm Optimization Using 2D Spatial Seperable Convolution"*
- [5] <https://www.mathworks.com/help/images/block-processing-large-images.html>